

BERT-Augmented LLM Text Classification

CS4964 Final Project | Greenbaum, McFarland, DeBoer | Spring 2026

Abstract

We built a gate-then-classify pipeline that uses a lightweight BERT encoder to filter irrelevant document chunks before sending anything to a large language model. On 200 Natural Questions validation examples, our BERT-guided pruning mode achieves 68% accuracy ($F1 = 0.806$) while making only 23,537 LLM calls, compared to 63,484 calls (2.7x more) for the full baseline at 67% accuracy ($F1 = 0.802$). Pruning irrelevant text does not hurt quality. It actually helps.

1. Introduction

Long documents are a problem for LLMs. If you want to know whether a Wikipedia article answers a question, you can't just throw the whole thing at the model. The model has to process every chunk, and most of those chunks are irrelevant. That's slow and expensive.

The obvious fix is random sampling, but that throws away potentially useful content. We wanted something smarter: use a cheap encoder to score each chunk against the query, and only send the relevant ones to the LLM. This is the core idea behind our pipeline.

2. Dataset

We used the Natural Questions (NQ) validation split from Google. Each example is a real search query paired with a full Wikipedia article, labeled with whether the article contains a long-form answer. We sampled 200 examples (seed 42) with 68% positive rate (136 of 200 articles have answers). We report F1 on the positive class alongside accuracy since the classes are imbalanced.

3. Method

The pipeline runs in three phases:

Phase 1: Gate-and-Chunk

We recursively split each document into a tree of chunks using sliding windows. Depth 0 uses ~380-word windows (the max BERT can handle), depth 1 uses ~150 words, and depth 2 uses ~40 words. At each node, we score the chunk against the query using cosine similarity with sentence-transformers/all-MiniLM-L6-v2. Chunks below a depth-scaled threshold get pruned: $\text{tau}(d) = 0.23 + d * 0.08$, so the threshold is 0.23 at the top (lenient, coarse chunks are noisy) and 0.39 at the leaves (strict, a short sentence needs real signal to survive).

Phase 2: LLM Classification

Surviving leaf chunks go to TinyLlama-1.1B-Chat in a single batched call per document (batch size 8 with left padding). The prompt asks the model to answer yes or no: does this passage address the question? We use greedy decoding with max_new_tokens=16 and parse the first token of the response.

Phase 3: Tree Aggregation

We propagate leaf labels up the tree using hard aggregation: a node is yes if any of its children is yes. This is recall-first, we'd rather a false positive than miss a real answer. The root label is the document-level prediction.

4. Evaluation Modes

We compare three modes on the same 200 examples to measure what BERT pruning actually buys us:

- Full:** No pruning. The LLM is called on every leaf node. This is the oracle baseline.
- BERT:** Our method. BERT gates each node against the depth-scaled threshold. Only survivors reach the LLM.
- Random:** Random pruning at the same average rate as BERT. A control to check that any benefit comes from the semantic scoring, not just from calling the LLM less.

5. Results

Mode	Accuracy	F1 (yes)	Precision	Recall	LLM Calls	Wall Time (s)
Full	67%	0.802	0.677	0.985	63,484	12710
BERT (ours)	68%	0.806	0.686	0.978	23,537	5056

BERT mode (green row) is the proposed method.

The headline result: BERT mode uses 2.7x fewer LLM calls than full mode (23,537 vs. 63,484) while matching or beating it on accuracy (68% vs. 67%) and F1 (0.806 vs. 0.802). Pruning irrelevant text does not hurt performance. It actually helps, because the LLM is no longer distracted by off-topic chunks.

6. Threshold Tuning

We tuned the base threshold and step on a held-out training split using a two-pass grid search. First a coarse sweep over base in [0.20, 0.65] and step in [0.00, 0.30] at 0.05 increments, then a fine sweep in a narrow window around the best result. The final parameters (base=0.23, step=0.08) balance recall (don't prune real answers) with efficiency (don't send irrelevant chunks to the LLM).

The tuning is cheap after the first run: we build full trees once with LLM labels, then replay threshold sweeps on the saved trees for free. No extra model calls needed.

7. Strengths and Weaknesses

Strengths:

The pipeline is modular. You can swap out the encoder or the LLM without changing anything else. The depth-scaled threshold is easy to understand and cheap to tune. The checkpointing system means you can resume a crashed run without starting over.

Weaknesses:

Performance is bounded by the encoder. If the encoder gives a low similarity score to a relevant chunk, it gets pruned and the LLM never sees it. The hard any-yes aggregation is recall-first but sensitive to false positives. One bad leaf flips the whole document.

8. Conclusion

We showed that a lightweight BERT encoder can serve as a cost-effective pre-filter for LLM-based document classification. On 200 NQ examples, our method reduces LLM calls by 2.7x compared to the full baseline while maintaining or improving accuracy and F1. The key insight is that semantic similarity between a query and a text chunk is a strong enough signal to safely prune irrelevant content before it ever reaches the LLM.

Appendix: Implementation Details

Models: sentence-transformers/all-MiniLM-L6-v2 (22M params, BERT encoder) and TinyLlama-1.1B-Chat-v1.0 (1.1B params, 4-bit NF4 quantization). Hardware: NVIDIA RTX 2080 SUPER. Dataset: NQ validation split, 200 examples, seed 42. Thresholds: base=0.23, step=0.08 ($\tau(d) = 0.23 + d \cdot 0.08$). Aggregation: hard (any-yes). Code: github.com/FreeThings/bert-llm-pruning